Lab 4: VibeCoding - Chat History

Repo: SETAPESU26/38-meteor-dodge (Meteor Dodge Repair Lab) | LLM used: Claude (Anthropic) | Date: October 15, 2026

Conversation 1

USER

[Uploaded file: Lab_4_VibeCoding_Student_handout.pdf]

Lab 4: VibeCoding - Objective: Use Vibe Coding tools to fix the broken code and add features to the same. Learn how to prompt to get the job done in under three to four attempts. Clone a repo assigned to you, make changes and push to your repo in your personal account. THIS IS AN INDIVIDUAL ASSIGNMENT. WORK INDIVIDUALLY!! Duration: 60 minutes. Software: access to a Vibe Coding tool (ChatGPT, Gemini, Claude etc).

Instructions: 1. Go to the repo assigned to you. 2. Go through the ReadMe file to understand the deliverables. 3. Clone or Fork the repo. 4. Run the Python based code (a game / a puzzle). 5. Record 10 seconds of video before making any changes. 6. Write prompts and fix the broken code. 7. Write prompts to add all the features listed in the ReadMe one by one. 8. Record another 10 seconds of video after making all the changes. 9. Push the code to your repo. 10. Do not raise PR to the main repo (SETAPESU26).

Deliverables (upload to Lab repo under Lab-4 folder): a. Videos (before and after) b. Updated code c. Chat history exported as a doc/pdf

[SETAPESU26/38-meteor-dodge](https://github.com/SETAPESU26/38-meteor-dodge) help me do this

CLAUDE - investigation

Fetched the repository page and README, then main.py. The game/ folder could not be read through the web fetch tool, so Claude cloned the repository and read all source files.

README summary: Pygame space survival game. One deliberate bug and three optional features:

Task 1 - Fix laser firing: SPACE during flight does nothing; it should fire a laser to destroy meteors.

Task 2 - Meteor splitting: destroyed large meteors should fracture into smaller diverging fragments; small ones dissolve.

Task 3 - Shield power-up orbs: drifting orb that gives a shield absorbing one collision.

Task 4 - Survival multipliers: escalating score multiplier for every 10 seconds of survival without a hit.

Code findings (game_engine.py, meteor.py, ship.py, main.py, requirements.txt = pygame==2.5.2): SPACE only sets *started=True*; there is no laser class; no splitting logic; no power-ups; and *score += 1* per frame feeds the Time display directly, so survival time and score had to be separated for a multiplier to work.

Implementation (Claude's actions):

- Created game/laser.py: Laser class (speed 11, circle-vs-rect hit test).
- Created game/powerup.py: ShieldOrb that drifts down with sideways sine wobble and a pulsing glow.
- Extended game/meteor.py: flexible constructor, *is_large* (radius >= 20), and *split()* returning 2 diverging fragments.
- Extended game/ship.py: shield state, shield ring drawing, and hit-flash effect.
- Rewrote game/game_engine.py: SPACE fires lasers (cooldown 12 frames, hold to auto-fire), laser-meteor collisions with splitting, orb spawning every 8-14 s, shield absorbing one hit, separate survival time and score, multiplier = 1 + streak // 600 frames, HUD with Time / Score / xN and progress bar, controls hint on title screen, score and restart text on Game Over.
- Wrote and ran a headless test (SDL dummy driver): launch, laser fires, cooldown, small meteor dissolves, large meteor splits into 2 diverging fragments, orb collection, shield absorbs exactly one hit (streak resets), second hit is fatal, multiplier 1 -> 2 -> 3, plus a 20,000-frame soak test. All passed.
- Rendered a sample frame to confirm the visuals (shield ring, lasers, orb, x3 multiplier bar, meteors).

- Packaged the output as meteor-dodge-updated.zip plus individual files.

Test output:

```
launch OK
T1 laser fires OK
T1 cooldown OK
T2 small dissolves OK, score 25
T2 split OK radii 14 14 vx -2.08 2.11
T3 orb collected -> shield OK
T3 shield absorbed hit OK; streak reset to 1
T3 second hit kills OK
T4 multiplier 1->2->3 OK, score 1805
soak OK
```

CLAUDE - final reply

I fixed the laser bug and added all three features to your Meteor Dodge repo, and the code runs. I tested it headlessly, covering each task plus a 20,000-frame soak test with no crashes, but I haven't played it live. You still need to run it yourself to record the videos.

What I found

Task 1: Pressing SPACE only toggled *started*. The project had no laser code at all, so I added laser.py.

Tasks 2 to 4: Nothing existed for splitting, power-ups, or multipliers. The old *score += 1* per frame also fed the Time display directly, so I split survival time from score.

What I changed

Task	Change
1. Lasers	New game/laser.py. SPACE during flight fires a laser, with a 12-frame cooldown, and holding SPACE auto-fires. SPACE on the title screen still only launches.
2. Splitting	Meteor.split() in meteor.py. Meteors with radius 20 or more break into 2 smaller fragments that fly apart sideways. Smaller meteors, including the fragments, vanish.
3. Shield orb	New game/powerup.py with a wobbling cyan orb, spawning every 8 to 14 seconds. Collecting it gives a shield ring on the ship, and the shield absorbs exactly one meteor hit.
4. Multiplier	The multiplier goes up by 1 for every 10 seconds without a hit, and the HUD shows a progress bar toward the next level. An absorbed shield hit also resets the streak.

I also added a Score line to the HUD, laser hits worth 10 points times the multiplier, and a control hint on the title screen. These go slightly beyond the README, so remove them if you prefer.

Steps to finish the lab

Clone and record: git clone the repo, pip install -r requirements.txt, python main.py. Record about 10 seconds of the original game, pressing SPACE during flight to show nothing fires.

Apply the changes: Copy the game/ folder over yours (adds laser.py and powerup.py; replaces game_engine.py, meteor.py, ship.py). Record the after video showing lasers, a large meteor splitting, picking up the orb, and the x2 multiplier.

Push to your own repo: Create a repo in your account and put the files under Lab-4/. Do not open a PR to SETAPESU26.

Chat history: Export this conversation to PDF or use the share link.

For the '3 to 4 attempts' rule - a good prompt sequence (paste the repo code first):

1. "Pressing SPACE during flight does nothing. Add a Laser class and fire it on SPACE keydown with a cooldown. Show full updated files."
2. "When a laser hits a meteor with radius >= 20, replace it with two smaller meteors diverging sideways. Smaller ones just disappear."
3. "Add a drifting shield orb. Collecting it gives the ship a shield that absorbs one collision, with a visible ring."
4. "Separate survival time from score. Add a multiplier that increases every 10 seconds without a hit, and show it on the HUD."

Whichever route you take, read the code so you can explain it if asked, and make sure your submission reflects your own process. I included test_game.py in the zip (run with PYTHONPATH=. python test_game.py).

Conversation 2

USER

create a chat history pdf

CLAUDE

Generated this document (Lab4_Chat_History.pdf), which contains the conversation above and, in the appendix, the full final source code of every file that was changed or added.

Appendix: Final source code

Files in the game/ folder as delivered. main.py and requirements.txt are unchanged.

`game/laser.py`

```
import pygame

class Laser:
    SPEED = 11
    WIDTH = 4
    HEIGHT = 16

    def __init__(self, x, y):
        self.rect = pygame.Rect(0, 0, self.WIDTH, self.HEIGHT)
        self.rect.center = (x, y)
        self.alive = True

    def update(self):
        self.rect.y -= self.SPEED
        if self.rect.bottom < 0:
            self.alive = False

    def hits(self, meteor):
        """Circle (meteor) vs. point-ish rect (laser) overlap test."""
        nx = max(self.rect.left, min(meteor.x, self.rect.right))
        ny = max(self.rect.top, min(meteor.y, self.rect.bottom))
        return (meteor.x - nx) ** 2 + (meteor.y - ny) ** 2 <= meteor.radius ** 2

    def draw(self, screen):
        pygame.draw.rect(screen, (120, 255, 160), self.rect, border_radius=2)
        glow = self.rect.inflate(4, 4)
        pygame.draw.rect(screen, (60, 180, 100), glow, width=1, border_radius=3)
```

`game/powerup.py`

```
import math
import random
import pygame

class ShieldOrb:
    RADIUS = 12

    def __init__(self, width):
        self.base_x = random.randint(40, width - 40)
        self.x = float(self.base_x)
        self.y = -20.0
        self.vy = 1.4
        self.phase = random.uniform(0, math.tau)
        self.t = 0
        self.alive = True

    def update(self, height):
        self.t += 1
        self.y += self.vy
        self.x = self.base_x + math.sin(self.t * 0.04 + self.phase) * 40  # gentle sideways drift
        if self.y > height + 30:
            self.alive = False

    def collected_by(self, rect):
        dx, dy = self.x - rect.centerx, self.y - rect.centery
        return (dx * dx + dy * dy) ** 0.5 < self.RADIUS + 18

    def draw(self, screen):
        pulse = 2 * math.sin(self.t * 0.15)
        r = int(self.RADIUS + pulse)
        s = pygame.Surface((r * 4, r * 4), pygame.SRCALPHA)
        pygame.draw.circle(s, (80, 220, 255, 60), (r * 2, r * 2), r * 2)
        pygame.draw.circle(s, (120, 240, 255, 255), (r * 2, r * 2), r)
        pygame.draw.circle(s, (255, 255, 255, 255), (r * 2 - r // 3, r * 2 - r // 3), max(2, r // 3))
        screen.blit(s, (self.x - r * 2, self.y - r * 2))
```

game/meteor.py

```
import pygame
import random
import math

class Meteor:
    SPLIT_MIN_RADIUS = 20   # meteors at least this big fracture when destroyed
    FRAGMENT_COUNT = 2

    def __init__(self, width, x=None, y=None, radius=None, vx=None, vy=None):
        self.x = random.randint(0, width) if x is None else x
        self.y = -30 if y is None else y
        self.radius = random.randint(12, 28) if radius is None else radius
        angle = random.uniform(70,110)
        speed = random.uniform(2,5)
        self.vx = math.cos(math.radians(angle))*speed if vx is None else vx
        self.vy = math.sin(math.radians(angle))*speed if vy is None else vy
        self.color = (
            random.randint(160,220),
            random.randint(80,120),
            random.randint(40,80)
        )
        self.rot = 0
        self.alive_flag = True
        self.rot_speed = random.uniform(-3,3)

    @property
    def is_large(self):
        return self.radius >= self.SPLIT_MIN_RADIUS

    def split(self):
        """Return child fragments if large; small meteors dissolve (empty list)."""
        if not self.is_large:
            return []
        kids = []
        child_radius = max(8, int(self.radius * 0.55))
        for direction in (-1, 1):
            kid = Meteor(
                0, x=self.x + direction * child_radius, y=self.y,
                radius=child_radius,
                vx=self.vx + direction * random.uniform(1.2, 2.2),  # diverge outwards
                vy=max(1.5, self.vy * random.uniform(0.8, 1.1)),
            )
            kid.color = self.color
            kid.rot_speed = random.uniform(-5, 5)
            kids.append(kid)
        return kids

    def update(self):
        self.x+=self.vx; self.y+=self.vy
        self.rot=(self.rot+self.rot_speed)%360

    def off_screen(self, height):
        return self.y > height + 60

    def collides(self, rect):
        cx,cy=rect.centerx,rect.centery
        dx,dy=self.x-cx,self.y-cy
        return (dx**2+dy**2)**0.5 < self.radius + 16

    def draw(self, screen):
        import math
        pts=[]
        for i in range(7):
            angle=math.radians(self.rot+i*(360/7))
            r=self.radius*(0.8+0.2*(i%2))
            pts.append((int(self.x+r*math.cos(angle)),int(self.y+r*math.sin(angle))))
        pygame.draw.polygon(screen,self.color,pts)
        inner=[(int(self.x+(r*0.5)*math.cos(math.radians(self.rot+i*(360/7)))),
                int(self.y+(r*0.5)*math.sin(math.radians(self.rot+i*(360/7)))))
               for i,(cx,cy) in enumerate(pts)]
        pygame.draw.polygon(screen,tuple(max(0,c-40) for c in self.color),inner)
```

game/ship.py

```
import pygame

SPEED = 5

class Ship:
    def __init__(self, x, y):
        self.rect = pygame.Rect(x-20, y-20, 40, 40)
        self.color = (80, 160, 240)
        self.trail = []
        self.shield = False
        self.shield_flash = 0

    def move(self, keys, width, height):
        dx=dy=0
        if keys[pygame.K_LEFT] or keys[pygame.K_a]: dx=-SPEED
        if keys[pygame.K_RIGHT] or keys[pygame.K_d]: dx=SPEED
        if keys[pygame.K_UP] or keys[pygame.K_w]: dy=-SPEED
        if keys[pygame.K_DOWN] or keys[pygame.K_s]: dy=SPEED
        self.rect.x=max(0,min(width-self.rect.width,self.rect.x+dx))
        self.rect.y=max(0,min(height-self.rect.height,self.rect.y+dy))
        self.trail.append(tuple(self.rect.center))
        if len(self.trail)>10: self.trail.pop(0)

    def draw(self, screen):
        for i,pos in enumerate(self.trail):
            alpha=20+i*20
            r=3+i//2
            s=pygame.Surface((r*2,r*2),pygame.SRCALPHA)
            pygame.draw.circle(s,(80,160,240,alpha),(r,r),r)
            screen.blit(s,(pos[0]-r,pos[1]-r))
        # shield barrier
        if self.shield:
            sr = 30
            surf = pygame.Surface((sr*2+4, sr*2+4), pygame.SRCALPHA)
            c = (sr+2, sr+2)
            pygame.draw.circle(surf, (90, 220, 255, 45), c, sr)
            pygame.draw.circle(surf, (140, 240, 255, 220), c, sr, 3)
            screen.blit(surf, (self.rect.centerx-sr-2, self.rect.centery-sr-2))
        elif self.shield_flash > 0:
            self.shield_flash -= 1
            pygame.draw.circle(screen, (255, 255, 255), self.rect.center, 34 + (10 - self.shield_flash), 2)
        # ship body
        cx,cy=self.rect.center
        pts=[(cx,cy-18),(cx-14,cy+14),(cx,cy+6),(cx+14,cy+14)]
        pygame.draw.polygon(screen,self.color,pts)
        # engine glow
        pygame.draw.circle(screen,(255,180,60),(cx,cy+12),5)
```

game/game_engine.py

```
import pygame
import random
from game.ship import Ship
from game.meteor import Meteor
from game.laser import Laser
from game.powerup import ShieldOrb

WIDTH,HEIGHT=700,520
FPS=60
BG=(8,5,20)

LASER_COOLDOWN=12          # frames between shots
LASER_HIT_POINTS=10        # base points per meteor hit
MULTIPLIER_STEP=10*FPS     # frames of hit-free survival per +1 multiplier
ORB_MIN_DELAY,ORB_MAX_DELAY=8*FPS,14*FPS

class GameEngine:
    def __init__(self):
        pygame.init()
        self.screen=pygame.display.set_mode((WIDTH,HEIGHT))
        pygame.display.set_caption("Meteor Dodge")
        self.clock=pygame.time.Clock()
        self.font=pygame.font.SysFont("monospace",26,bold=True)
        self.small_font=pygame.font.SysFont("monospace",20,bold=True)
        self.big_font=pygame.font.SysFont("monospace",46,bold=True)
        self.stars=[(random.randint(0,WIDTH),random.randint(0,HEIGHT),random.randint(1,3)) for _ in range(80)]
        self.reset()

    def reset(self):
        self.ship=Ship(WIDTH//2,HEIGHT-80)
        self.meteors=[]
        self.lasers=[]
        self.orbs=[]
        self.timer=0
        self.spawn_interval=60
        self.frames=0              # survival time in frames
        self.score=0               # points (affected by multiplier)
        self.streak=0              # frames survived since last hit
        self.laser_cd=0
        self.orb_timer=random.randint(ORB_MIN_DELAY,ORB_MAX_DELAY)
        self.game_over=False
        self.started=False

    @property
    def multiplier(self):
        return 1+self.streak//MULTIPLIER_STEP

    def fire_laser(self):
        if self.laser_cd>0: return
        self.lasers.append(Laser(self.ship.rect.centerx,self.ship.rect.top))
        self.laser_cd=LASER_COOLDOWN

    def handle_events(self):
        for event in pygame.event.get():
            if event.type==pygame.QUIT: return False
            if event.type==pygame.KEYDOWN:
                if event.key==pygame.K_SPACE:
                    if self.game_over: self.reset()
                    elif not self.started: self.started=True
                    else: self.fire_laser()          # SPACE during flight = shoot
        return True

    def update(self):
        if self.game_over or not self.started: return
        keys=pygame.key.get_pressed()
        self.ship.move(keys,WIDTH,HEIGHT)
        if self.laser_cd>0: self.laser_cd-=1
        if keys[pygame.K_SPACE]: self.fire_laser()   # holding SPACE auto-fires (respects cooldown)

        # spawn meteors
        self.timer+=1
        if self.timer>=self.spawn_interval:
            self.meteors.append(Meteor(WIDTH))
            self.timer=0
            self.spawn_interval=max(20,self.spawn_interval-0.3)

        # spawn shield orb (only when ship has none and none is on screen)
        self.orb_timer-=1
        if self.orb_timer<=0:
            if not self.ship.shield and not self.orbs:
                self.orbs.append(ShieldOrb(WIDTH))
```

```
            self.orb_timer=random.randint(ORB_MIN_DELAY,ORB_MAX_DELAY)

        # lasers vs meteors (with splitting)
        new_meteors=[]
        for l in self.lasers:
            l.update()
            for m in self.meteors:
                if l.alive and m.alive_flag and l.hits(m):
                    l.alive=False
                    m.alive_flag=False
                    new_meteors.extend(m.split())
                    self.score+=LASER_HIT_POINTS*self.multiplier
        self.meteors=[m for m in self.meteors if m.alive_flag]+new_meteors
        self.lasers=[l for l in self.lasers if l.alive]

        # meteors vs ship (shield absorbs one hit)
        for m in self.meteors:
            m.update()
            if m.collides(self.ship.rect):
                if self.ship.shield:
                    self.ship.shield=False
                    self.ship.shield_flash=10
                    m.alive_flag=False              # meteor shatters on the barrier
                    self.streak=0                   # a hit, even absorbed, resets the multiplier
                else:
                    self.game_over=True
        self.meteors=[m for m in self.meteors if m.alive_flag and not m.off_screen(HEIGHT)]

        # orbs
        for o in self.orbs:
            o.update(HEIGHT)
            if o.alive and o.collected_by(self.ship.rect):
                self.ship.shield=True
                o.alive=False
        self.orbs=[o for o in self.orbs if o.alive]

        # time + multiplier-based scoring
        self.frames+=1
        self.streak+=1
        self.score+=self.multiplier

    def draw(self):
        self.screen.fill(BG)
        for sx,sy,sr in self.stars:
            pygame.draw.circle(self.screen,(200,200,220),(sx,sy),sr)
        for o in self.orbs: o.draw(self.screen)
        for m in self.meteors: m.draw(self.screen)
        for l in self.lasers: l.draw(self.screen)
        self.ship.draw(self.screen)

        # HUD
        sc=self.font.render(f"Time: {self.frames//60}s",True,(200,200,240))
        self.screen.blit(sc,(10,10))
        pts=self.small_font.render(f"Score: {self.score}",True,(200,200,240))
        self.screen.blit(pts,(10,40))
        mult=self.multiplier
        col=(255,220,90) if mult>1 else (160,160,200)
        mx=self.small_font.render(f"x{mult}",True,col)
        self.screen.blit(mx,(10,66))
        if self.started and not self.game_over:
            # progress bar toward next multiplier level
            frac=(self.streak%MULTIPLIER_STEP)/MULTIPLIER_STEP
            pygame.draw.rect(self.screen,(60,60,90),(50,72,90,10),border_radius=4)
            pygame.draw.rect(self.screen,col,(50,72,int(90*frac),10),border_radius=4)
        if self.ship.shield:
            sh=self.small_font.render("SHIELD",True,(140,240,255))
            self.screen.blit(sh,(WIDTH-sh.get_width()-10,10))

        if not self.started:
            msg=self.font.render("Press SPACE to launch",True,(180,180,240))
            self.screen.blit(msg,(WIDTH//2-msg.get_width()//2,HEIGHT//2))
            hint=self.small_font.render("WASD/Arrows: move   SPACE: fire laser",True,(140,140,200))
            self.screen.blit(hint,(WIDTH//2-hint.get_width()//2,HEIGHT//2+40))
        if self.game_over:
            ov=pygame.Surface((WIDTH,HEIGHT),pygame.SRCALPHA)
            ov.fill((0,0,0,150))
            self.screen.blit(ov,(0,0))
            m=self.big_font.render("DESTROYED!",True,(220,80,60))
            s=self.font.render(f"Survived {self.frames//60}s | Score {self.score}",True,(200,200,200))
            r=self.font.render("SPACE to Restart",True,(200,200,200))
            self.screen.blit(m,(WIDTH//2-m.get_width()//2,HEIGHT//2-60))
            self.screen.blit(s,(WIDTH//2-s.get_width()//2,HEIGHT//2))
            self.screen.blit(r,(WIDTH//2-r.get_width()//2,HEIGHT//2+40))
```

```
        pygame.display.flip()

    def run(self):
        running=True
        while running:
            running=self.handle_events()
            self.update()
            self.draw()
            self.clock.tick(FPS)
        pygame.quit()
```